

Big Data Analytics System E-Commerce Platform

Technical Report

MongoDB · HBase · Apache Spark

Name

Wayne Rubangisa

Student ID

101220

Course

Big Data Analytics

Programme

MSc in Big Data Analytics

Institution

Adventist University of Central Africa (AUCA)

Technologies

MongoDB 7.0 · HBase (Thrift API) · PySpark 3.5.0 · Docker

Big Data Analytics System: E-Commerce Platform

Technical Report: MongoDB, HBase, and Apache Spark over a shared, reproducible e-commerce dataset

1. System Architecture Overview

The system is built around one principle: a single generated dataset, consumed independently by three different storage and processing technologies, each containerised with Docker. `dataset_generator.py` produces all source data with fixed random seeds (`np.random.seed(42)`, `random.seed(42)`, `Faker.seed(42)`), guaranteeing reproducible output across runs.

MongoDB (`mongo:7.0`, container `mongo-ecommerce`) stores the product catalogue, user profiles, and transaction history as JSON documents. **HBase** (`harisekhon/hbase`, container `hbase-project`) stores time-series session and product performance metrics, accessed via the Thrift API on port 9090. **Spark** (`jupyter/pyspark-notebook`) reads the generated JSON files directly from disk, it does not connect to MongoDB or HBase at runtime. Integration across the three systems therefore happens at the data level: all three technologies consume the same generated dataset (2,000 users, 1,500 products, 40,000 transactions, 150,000 sessions over a 90-day window) independently rather than through live connectors.

2. Data Modeling Decisions and Rationale

Every embed-versus-reference decision was made against four criteria: ownership/locality, duplication risk, growth boundedness, and, for transactional data, historical accuracy (must a value stay frozen even if the live source later changes).

In **MongoDB**, `price_history` is embedded in each product (owned, bound to 0–3 entries, never queried independently), while `category_id` is a reference, since embedding would duplicate category data across roughly 60 products on average. The most consequential decision is in **transactions**: line items embed a hybrid structure, a `product_id` reference plus a `unit_price/subtotal` snapshot, rather than a live reference, because a later price change must not silently rewrite what a customer historically paid. **Sessions** embed `page_views` (one bounded browsing journey) but are themselves referenced from users via `user_id` rather than embedded, since one user's sessions grow unboundedly over an account's lifetime and risk the 16MB document ceiling. A cached user purchase-summary field was deliberately rejected in favour of

computing it on demand, avoiding a denormalised value that could silently desynchronise from the source of truth.

Relationship	Decision	Justification
Product → price_history	Embed	Owned, bounded, never queried independently
Product → category	Reference	Shared across ~60 products; avoids duplication
Transaction → line items	Embed (hybrid snapshot)	Historical accuracy of recorded price
Transaction → user	Reference	One user generates many transactions
Session → page_views	Embed	Bounded, always read together
User → sessions	Reference	Unbounded growth; document-size risk

Index decisions were backed by `.explain("executionStats")` evidence rather than assumed: a query for one user's transactions against an unindexed `transactions.user_id` examined all 40,000 documents (`totalDocsExamined: 40000`); after creating `{ user_id: 1 }`, the same kind of query for a different user examined only the 9 matching documents. The `sessions` collection, the highest-write-volume collection, was deliberately left with only its default index, while the small, read-heavy `products` collection was indexed on `product_id` to support the `$lookup` join used in revenue reporting, reflecting a general write/read trade-off: every index adds cost on writes, worth paying only where reads and joins dominate.

In **HBase**, the `sessions` table uses row key `user_id#reverse_timestamp`, `user_id` first so all of a user's sessions sit contiguously on disk for a fast prefix scan, and the timestamp reversed (`999999999999 - ts`) so the dominant most-recent-first access pattern requires no extra sort. Four column families (`info`, `device`, `geo`, `activity`) physically separate data by access pattern, so a device-breakdown report never touches the `geo` or `activity` files. The product performance table uses `product_id#date` with a normal (non-reversed) date, since that analysis is chronological, a reversed key would invert the trend line, and a single `metrics` family, since `view_count`, `cart_add_count`, and `purchase_count` are always read together as one daily snapshot. Prefix scans on both tables return results from the 150,003-row `sessions` table and 208,286-row product-performance table in well under a second (for example, 80 sessions for one user in 0.67 seconds, and 90 days of one product's metrics in 0.13 seconds); a query filtering on a non-key field (`conversion_status`) requires a full table scan, illustrating HBase's core constraint that only the row key is indexed. A secondary index table keyed on the filtered field is the standard remedy, discussed further in Section 10.

3. Spark Processing Pipelines

PySpark 3.5.0 runs in the `jupyter/pyspark-notebook` container, reading all JSON files mounted at `/home/jovyan/data/`. Because the source files use a single-array JSON format rather than JSON Lines, `.option("multiLine", True)` is required on every read, omitting it does not raise an error but silently produces a `_corrupt_record` column. Cleaning resolved five issues:

- Timestamp strings (`registration_date`, `last_active`, transaction timestamp) are inferred as plain strings, requiring explicit `to_timestamp()` / `date_format()` calls.
- Nested fields (`geo_data`, `device_profile`) are structs, accessed via dot notation or `withColumn + col()`.
- An ambiguous `subtotal` column exists at both transaction and line-item level; exploding items without renaming raises `AMBIGUOUS_REFERENCE`. Resolved by aliasing to `order_subtotal` and `line_subtotal`.
- Schema inference misreads `cart_contents` as a 1,500-field struct instead of a map; resolved with an explicit `MapType` schema.
- ~20% of transactions carry a null `session_id` (generated outside the session flow); handled with `fillna({"session_id": "no_session"})`.

Cleaned DataFrames were registered as Spark SQL temp views. Representative queries include a multi-source join of users and transactions for per-user total spend, a CTE/scalar-subquery identifying users above the population average spend (top result: user_001552 at \$43,634), and a top-5-countries-by-revenue join (led by KM at \$424,894).

4. Integrated Analytics Workflows

Two analyses integrate data across the generated sources within Spark. **Cohort analysis** groups users by `registration_month` and transactions by `activity_month`, joins on `user_id`, and computes `months_since_registration` to pivot into a cohort revenue table (rows = registration month, columns = months since registration, spanning registration months 2025-09 through 2026-03). Revenue is consistently highest at months 3–5 across all cohorts, reflecting a ramp-up period after users first register. The October 2025 cohort shows the highest single-month revenue of any cohort (\$2,804,777 at month 7), while the March 2026 cohort shows the highest first-month revenue of any cohort (\$589,816 at month 0), suggesting improving acquisition quality over time. A diagonal band of nulls in the pivoted table reflects the

dataset's fixed 90-day observation window rather than missing data: earlier cohorts simply have more months of history visible than recent ones.

Customer Lifetime Value (CLV) integrates all three generated sources, transactions (spend, order count), sessions (frequency, conversion rate), and users (registration, country), joined on `user_id` (confirmed: all 2,000 users present in all three). The score is:

$$\text{CLV score} = \text{total_spend} \times (1 + \text{conversion_rate}) \times \log(\text{total_sessions} + 1)$$

Users were ranked by `percent_rank()` and bucketed into High (top 20%), Mid (next 30%), and Low (bottom 50%) value segments:

Segment	Users	Avg CLV Score	Avg Spend	Avg Conversion
High Value	400	159,557	\$29,566	23.6%
Mid Value	600	120,993	\$23,073	21.1%
Low Value	1,000	83,813	\$16,438	18.6%

5. Scalability Considerations

MongoDB scales horizontally via sharding, with `user_id` as a natural shard key for users and transactions, routing user-filtered queries directly to the correct shard. **HBase** scales by adding RegionServers, which triggers automatic region redistribution; the sessions table's high-cardinality `user_id` prefix avoids write hotspotting. **Spark** scales by adding workers, the single-node notebook container would be replaced by a multi-node cluster (Standalone or Kubernetes) in production, with no code changes required since DataFrame partitioning is transparent to the API.

6. Technology Selection Justification

MongoDB fits the catalogue, profile, and transaction data because these entities are naturally document-shaped with variable nested structures (price history, line items, geo data) and relationships that do not require full relational joins; its aggregation pipeline was sufficient for the required analytics. **HBase** fits session and product-metric data because of three properties: high, continuous write volume; fixed, known query patterns (always by user or product, optionally time-bounded); and scale (150,000+ rows here, millions in production), its MemStore-then-flush write path and row-key-encoded query dimensions are purpose-built for

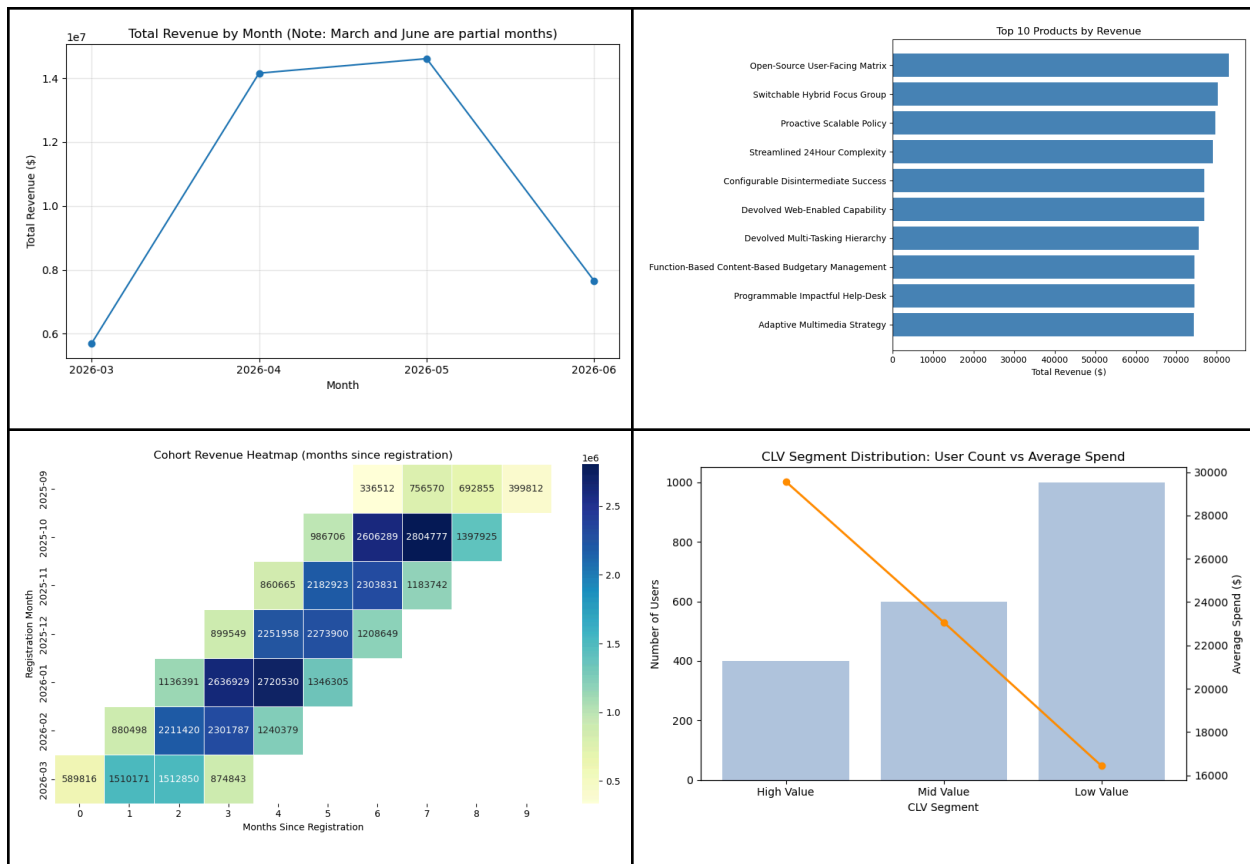
this. **Spark** fits the cross-cutting batch analytics (cleaning, cohort analysis, CLV) because its DataFrame API and lazy evaluation allow query-plan optimization before execution, and the same code scales from a single-node notebook to a distributed cluster without modification.

7. Key Findings and Business Insights

- **No true low-engagement segment exists:** every one of the 2,000 users made at least 5 transactions in the observation window, so a "re-engage inactive users" campaign has no zero-purchase target audience in this dataset; the nearest candidate is the 5–14-transaction group (244 users, 12.2%).
- **Revenue concentration:** the top 5 products by revenue each generated \$76,800–\$82,900, led by prod_00724 ("Open-Source User-Facing Matrix") at \$82,913 across 168 units.
- **Cohort ramp-up:** spending peaks at months 3–5 after registration across every cohort, and first-month revenue has trended upward in more recent cohorts (e.g., the March 2026 cohort's \$589,816 is the highest first-month figure of any cohort), suggesting improving acquisition quality over time.
- **Value and engagement move together:** the High Value CLV segment (top 20% of users) has both the highest average spend and the highest conversion rate, rather than trading one off against the other, supporting concentrating retention spend on this segment specifically.
- **Status filtering matters:** the dataset defines no `cancelled/failed` transaction status; filtering to be `completed` only would have silently excluded 36.9% of transactions from every revenue figure.

8. Visualization of Results

Four charts support the findings above. Final chart images are inserted by the project team from the Spark notebook's matplotlib/seaborn output;



9. Methodology Explanation for Key Analyses

Before any revenue analysis, the assumption that only status: "completed" transactions count as valid revenue was tested directly against the data (a \$group by status) rather than assumed, since no cancelled/failed status exists in this dataset; all four statuses were therefore included in every revenue figure. **Product revenue ranking** (MongoDB) uses \$unwind to expand transactions into 85,799 line items, \$group to sum revenue and units per product, then \$lookup to resolve product names, combining array deconstruction, cross-document aggregation, and a cross-collection join. **User segmentation** uses \$bucket rather than a raw sort, since an un-bucketed ranking surfaces extreme outliers (the most frequent buyer made 38 transactions) and obscures the true population shape. **Cohort analysis** (Spark) joins users to transactions on user_id, derives months_since_registration as an integer month difference, and pivots into a matrix, index decisions for the underlying MongoDB queries were verified with .explain("executionStats") rather than assumed (e.g., an index on transactions.user_id reduced documents examined from 40,000 to the exact 9 matching a given user). **CLV** combines all three sources via an inner join and ranks users with the percent_rank() window function before

bucketing into segments, so segment boundaries reflect the dataset's actual score distribution rather than fixed thresholds.

10. Limitations and Future Work

- The 90-day observation window limits cohort analysis to 9 months of visible history, introducing survivorship bias when comparing early vs. recent cohorts.
- Spark does not connect to MongoDB or HBase at runtime; integration is at the data-file level. The MongoDB Spark Connector and HBase-Spark Connector (SHC) were considered but excluded given configuration complexity relative to available time.
- The HBase sessions table supports only row-key queries; filtering by `conversion_status` requires a full scan. A secondary index table keyed on `conversion_status` would resolve this.
- The CLV formula is a heuristic proxy, not a formal financial model, it omits discount rate, churn probability, and acquisition cost, none of which are available in the generated dataset.
- MongoDB and HBase containers run without authentication, appropriate for local development but not production.
- `categories.json` was not loaded into MongoDB; category-based analytics is currently limited to the opaque `category_id` rather than a resolved name.